

DeepMindQ

Full Platform Audit Report

Pre-WI-18 Architecture, Product & Production Readiness Review

Audit Date	August 3, 2026
Codebase	Next.js 16 + Prisma + Neon PostgreSQL
Total Source Files	676 (.ts/.tsx)
Total Lines of Code	213,617
Audit Scope	12 Categories, Read-Only
Verdict	B) Advanced MVP

Table of Contents

1. Executive Summary	4
1.1 Platform Readiness Verdict	4
2. Codebase Architecture	6
2.1 Technology Stack	6
2.2 Code Volume Metrics	6
2.3 Architecture Patterns	6
2.4 Module Coupling	6
2.5 Key Findings	6
3. AI Intelligence Architecture	8
3.1 LLM Provider Architecture	8
3.2 Knowledge Graph	8
3.3 Hybrid Retrieval System	8
3.4 AI Memory System	8
3.5 Hallucination Prevention	8
3.6 Explainability & Recommendation	9
3.7 Key Findings	9
4. AI Integration End-to-End	10
4.1 Flow A: Company Intelligence Activation	10
4.2 Flow B: Intelligence Query (Alignment)	10
4.3 Flow C: Knowledge Graph Enrichment	10
4.4 Flow D: Feedback Learning Loop	10
5. Database Architecture	11
5.1 Schema Design	11
5.2 Index Coverage	11
5.3 Data Integrity Concerns	11
5.4 Key Findings	11
6. Data Intelligence Pipeline	12
6.1 Data Import	12
6.2 Intelligence Sources	12
6.3 Key Findings	12
7. Backend API Architecture	13
7.1 API Patterns	13
7.2 Authentication Coverage	13

7.3 Missing Infrastructure	13
7.4 Key Findings	13
8. UI/UX Analysis	14
8.1 Component Quality	14
8.2 Accessibility	14
8.3 Theme & Responsive	14
8.4 Key Findings	14
9. Security Analysis	15
9.1 Authentication	15
9.2 Critical Security Gaps	15
9.3 Key Findings	15
10. Performance & Scalability	16
10.1 Database Performance	16
10.2 Frontend Performance	16
10.3 Deployment	16
10.4 Key Findings	16
11. Testing	17
11.1 Test Quality	17
11.2 Critical Gaps	17
11.3 Key Findings	17
12. Product Readiness Scoring	18
12.1 Verdict: B) Advanced MVP	18
13. Final Recommendations	20
13.1 P0: Blocking (Must Fix Before Enterprise Deployment)	20
13.2 P1: High Priority (WI-18 Phase)	20
13.3 P2: Medium Priority (Post-WI-18)	20
13.4 Architecture Strengths to Preserve	21

1. Executive Summary

This report presents a comprehensive, read-only audit of the DeepMindQ platform conducted on August 3, 2026, immediately following the completion of the WI-17 Productization milestone. The audit covers 12 categories spanning architecture, AI intelligence, database design, API layer, UI/UX, security, performance, testing, and overall product readiness. Every finding in this report is backed by actual codebase evidence obtained through systematic file analysis, not assumptions or summaries from previous sessions.

DeepMindQ is an AI-powered Revenue Intelligence Platform built on Next.js 16, React 19, Prisma 6, and Neon PostgreSQL. It implements a sophisticated multi-layer AI architecture including a 6-step intelligence activation pipeline, a knowledge graph, hybrid retrieval with RRF score fusion, 4-layer AI memory, hallucination prevention, explainability engine, recommendation engine, and a feedback learning loop. The platform encompasses 676 source files totaling 213,617 lines of code, 92 database models, 239 API routes, and 68 screen components.

The audit reveals a platform that has achieved an **Advanced MVP** level of maturity. The AI architecture is sophisticated and well-designed, the database schema is comprehensive with exceptional index coverage, and the codebase demonstrates clean dependency graphs with no circular dependencies. However, critical gaps exist in production readiness: three core AI systems (knowledge graph, memory, hybrid retrieval) are entirely in-memory with no persistence, the security middleware is missing entirely, and the CI/CD pipeline is absent. These findings define the roadmap from WI-18 onwards.

1.1 Platform Readiness Verdict

Based on the comprehensive 12-section analysis, DeepMindQ is classified as **B) Advanced MVP**. This means the platform demonstrates a sophisticated and functional product with deep AI capabilities, a comprehensive feature set, and working end-to-end flows. However, it is not yet production-ready for enterprise deployment due to security gaps, scalability limitations from in-memory architecture, and insufficient test coverage. The platform would benefit from a focused WI-18 phase addressing production hardening before customer-facing beta deployment.

Category	Score / 10	Status
Architecture	7.5	Strong AI architecture, clean deps
AI Intelligence	7.0	Sophisticated but in-memory only
Database	7.0	92 models, 372 indexes, some anti-patterns
Data Pipeline	6.5	Comprehensive but dual-path complexity
Backend API	5.5	239 routes, inconsistent patterns
UI/UX	6.0	Feature-rich, accessibility gaps
Security	3.5	Missing middleware, CSRF non-functional
Performance	4.5	In-memory state, no CI/CD
Testing	4.0	66% modules untested, no CI
Product Readiness	5.8	Advanced MVP, not enterprise-beta

2. Codebase Architecture

The DeepMindQ codebase is organized as a hybrid layer-based and domain-based architecture within a Next.js 16 App Router project. The top-level structure follows a clear layer separation with `src/app/` for routes, `src/components/` for UI, `src/lib/` for business logic, `src/providers/` for React context, and `src/hooks/` for custom hooks. Within the library layer, domain-based grouping is used for AI modules (intelligence-sources, revenue-intelligence, research-engine, engines, scoring, workflow-engine), while 98 top-level files in `src/lib/` create a potential discoverability challenge as the codebase grows.

2.1 Technology Stack

The platform leverages a modern stack: Next.js 16.1.1 with React 19, TypeScript 5 with strict mode enabled, Prisma 6.19.3 with Neon PostgreSQL, Tailwind CSS 4, Zustand for client navigation state, and TanStack React Query v5 for server-state management. The AI layer uses the Z.ai SDK (z-ai-web-dev-sdk) alongside four free-tier LLM providers (NVIDIA NIM, Fireworks AI, Groq, Google Gemini) and Tavily for web search. Additional notable dependencies include Zod 4 for validation, Recharts for data visualization, Framer Motion for animations, and Sentry for error tracking.

2.2 Code Volume Metrics

676	213,617	239	92	68	66
Source Files	Lines of Code	API Routes	DB Models	Screens	Test Files

Table 2: Codebase Size Metrics

The codebase has significant hotspot files: `ai-agent-framework.ts` at 2,873 lines, `company-profile-screen.tsx` at 2,450 lines, `ai-evaluation-engine.ts` at 2,006 lines, and the alignment API route at 1,027 lines. These giant files suggest modules that should be decomposed into smaller, more maintainable units. The test-to-code ratio is approximately 9.8%, well below the industry norm of 20-40%.

2.3 Architecture Patterns

Data Layer: Prisma ORM is used exclusively with a singleton client pattern. There is no repository pattern or data access layer. 345 files import `db.ts` directly, creating tight coupling to the schema. **API Layer:** 239 Next.js App Router route handlers follow a consistent but manually-replicated pattern of auth check, body validation, Prisma query, and response. An `api-middleware.ts` was built but is effectively dead code, used by only 1 of 239 routes. **State Management:** Zustand manages only 8 navigation state fields; React Query handles all server state with 30-second stale time.

2.4 Module Coupling

The most heavily imported modules are: `date.ts` (389 importers), `db.ts` (345), `logger.ts` (304), `api-auth.ts` (214), `store.ts` (119), `types.ts` (107), and `utils.ts` (99). The dependency graph is cleanly layered with no circular dependencies detected. The AI configuration layer is well-contained with `ai-config.ts` having only 2 importers, accessed primarily through `llm-client.ts`. The `api-middleware.ts` is dead code, having been built but never adopted by the broader route ecosystem.

2.5 Key Findings

ID	Severity	Finding	Evidence
A1	HIGH	No data access layer	345 files import db.ts directly
A2	HIGH	ESLint effectively disabled	Nearly all rules set to "off"
A3	MEDIUM	Dead code: api-middleware.ts	Only 1 of 239 routes uses it
A4	MEDIUM	Flat src/lib/ with 98 files	Discoverability challenge
A5	LOW	Empty seed data files	10 JSON files at 0 bytes
A6	LOW	AuthProvider is a no-op	Renders only children

Table 3: Architecture Findings

3. AI Intelligence Architecture

The AI intelligence layer is the most sophisticated and differentiated component of the DeepMindQ platform. It implements a multi-layer architecture with a unified LLM client, comprehensive governance framework, knowledge graph, hybrid retrieval, 4-layer memory system, hallucination prevention, explainability engine, recommendation engine, and feedback learning loop. The architecture is well-designed with a non-throwing design philosophy where every function returns structured results rather than throwing exceptions, providing excellent resilience for AI operations that are inherently unpredictable.

3.1 LLM Provider Architecture

The platform supports four free-tier LLM providers routed through a sequential fallback chain: NVIDIA NIM (meta/llama-3.1-8b-instruct, first priority), Fireworks AI (llama-v3p3-70b-instruct), Groq (llama-3.3-70b-versatile), and Google Gemini (gemini-2.0-flash with a 4-model fallback chain). Tavily is used for web search with 1000 requests/month on the free tier. The unified LLM client (llm-client.ts, 595 LOC) provides three call paths: callAI() through the Z.ai SDK with quality gates, callLLM() through direct provider fallback, and revenueLLMCall() as a never-throws variant for narrative generation.

3.2 Knowledge Graph

The knowledge graph (ai-knowledge-graph.ts, 1,764 LOC) implements a full in-memory graph database with entity extraction, BFS/DFS traversal, relationship scoring, evidence chain construction, and graph-based recommendations. It supports 28 relationship types and uses 6 Map structures for nodes, edges, and various indexes. The graph architecture is well-designed with proper confidence propagation and hop penalty decay. However, the entire knowledge graph is stored in process memory with zero database persistence, meaning all graph data is lost on every Vercel serverless cold start. This is the single most critical architectural gap for production deployment.

3.3 Hybrid Retrieval System

The hybrid retrieval engine (ai-hybrid-retrieval.ts, 1,202 LOC) implements a 5-signal retrieval pipeline: Vector Search (TF-IDF cosine similarity), Keyword Search (BM25-style), Entity Matching (regex-based NER), Knowledge Graph traversal, and Recency Weighting (exponential decay with 90-day half-life). Score fusion uses Reciprocal Rank Fusion (RRF) with k=60 smoothing constant. The architecture is enterprise-grade in design. Like the knowledge graph, it is entirely in-memory with a 100,000 entry cap and LRU eviction, meaning the retrieval index is rebuilt from scratch on every cold start.

3.4 AI Memory System

The 4-layer AI memory system (ai-memory.ts, 1,205 LOC) provides Working (session), Conversation (history), Enterprise (company intel), and Institutional (organizational learning) layers. It supports versioning, tag-based indexing, scope filtering, keyword search with multi-signal scoring, memory consolidation, and time-based decay. The institutional memory layer is the most valuable for learning but is completely ephemeral, lost on every serverless cold start. There is no memory size limit, unlike the hybrid retrieval cap, meaning memory could grow unbounded within a single process lifetime.

3.5 Hallucination Prevention

A two-layer hallucination prevention system is implemented. The pre-generation layer (ai-governance.ts, 1,473 LOC) injects 15 mandatory rules into every LLM system prompt, enforces confidence gates per 50+ generation types, and applies staleness-based confidence modifiers. The post-generation layer (ai-hallucination-prevention.ts, 666 LOC) performs claim extraction via 8 regex patterns, citation verification, hedging detection, and specificity scoring. A notable concern is that the alignment check uses keyword overlap rather than semantic similarity, which misses paraphrased claims. Additionally, the hedging detection penalizes honest uncertainty language, creating a perverse incentive for overconfident AI output.

3.6 Explainability & Recommendation

The explainability engine (1,391 LOC) generates 6-section intelligence trails with evidence attribution, confidence breakdowns, and reasoning chain visualization. The recommendation engine (1,082 LOC) implements composite scoring across 8 data sources with evidence-backed reasons, risk identification, and recommended actions with timelines. Both modules demonstrate sophisticated AI product design. The feedback learning loop (feedback-learning-loop.ts) captures user verdicts through a 4-step pipeline: capture feedback, create institutional memory, generate learning events, and calibrate signal confidence. A critical gap is that the institutional memory created from feedback is stored in the in-memory system and lost on cold starts.

3.7 Key Findings

ID	Severity	Finding	Evidence
AI1	CRITICAL	KG/Memory/Retrieval: in-memory only	Data lost on every serverless cold start
AI2	HIGH	ModelRouter tier routing broken	orderedChain computed but never used
AI3	HIGH	Usage tracking always returns 0	Z.ai SDK path has zero token counts
AI4	MEDIUM	Hedging penalizes honest uncertainty	Creates perverse overconfidence incentive
AI5	MEDIUM	extractJSON() greedy regex	Fails on nested JSON objects
AI6	MEDIUM	Type collision: QualityReport	Two interfaces with same name
AI7	LOW	Embeddings are TF-IDF only	No neural embeddings for semantic search

Table 4: AI Architecture Findings

4. AI Integration End-to-End

Four complete AI integration flows were traced through actual code paths to validate end-to-end wiring and identify gaps. Each flow was followed from API entry point through all intermediate processing steps to the final output or persistence layer.

4.1 Flow A: Company Intelligence Activation

Triggered by POST /api/companies (line 268 of route.ts), which calls activateIntelligenceAsync() after company creation. This fires a non-blocking 6-step pipeline: (1) Entity Resolution via regex NER, (2) Knowledge Graph population with node/edge creation, (3) Retrieval Index population, (4) Memory creation for company and contacts, (5) Signal Extraction via dual Tavily web searches + LLM analysis, and (6) Confidence scoring. The success threshold is lenient (3 of 6 steps complete), and steps 1-4 operate on empty in-memory stores on cold starts. No deduplication mechanism prevents parallel activations from creating duplicate signals.

4.2 Flow B: Intelligence Query (Alignment)

GET /api/companies/[id]/alignment is a read-only composition layer that makes zero AI/LLM calls. It queries 5 database tables in parallel (company, signals, capabilities, contacts, validations), composes IntelligenceObjects with evidence state, temporal confidence, freshness scoring, and origin derivation. A pagination gap exists as all signals are fetched without limit. The deriveOrigin() function will throw on malformed URLs, and the feedback map uses a string key that could collide when artifactId is null. Temporal confidence uses estimated previous values rather than actual historical data.

4.3 Flow C: Knowledge Graph Enrichment

Knowledge graph enrichment occurs through activation Step 2 and the knowledge ingestion pipeline. Entity extraction uses regex-based NER mapped to graph types, with relationships inferred from co-occurrence using 12 hardcoded rules. Relationship confidence caps at 0.7 for company+person co-occurrence. A critical bug was found: intelligence-activation.ts line 266 uses the invalid relationship type "employed_by" instead of "WORKS_AT". The ingestion pipeline classifies every 5th chunk with AI to reduce cost, meaning classification quality degrades for long documents with varying topics.

4.4 Flow D: Feedback Learning Loop

POST /api/feedback validates submission (5 verdict types, 16 reason codes, 11 outcomes) and processes through 4 steps: (1) DB persistence to IntelligenceFeedback table, (2) institutional memory creation with importance/confidence scoring based on verdict, (3) learning event creation for significant outcomes, and (4) signal confidence calibration based on accumulated feedback patterns. The institutional memory from feedback is stored in the in-memory system only, making it ephemeral. There is no mechanism for users to flag AI-generated content quality issues separately from recommendation feedback.

5. Database Architecture

The database architecture is built on Neon PostgreSQL with Prisma 6.19.3 as the ORM. The schema comprises 92 models, 20 enums, 372 indexes, and 80 relation declarations organized across 21 domain categories. The index coverage is exceptional at approximately 4 indexes per model on average. However, several design concerns exist including a "god model" pattern with Company having 35+ back-relations, 18 String-stored JSON fields that bypass Prisma JSON operators, and 97 cascade delete relations that create amplified deletion chains.

5.1 Schema Design

Models are categorized across: CRM/Company (8 models), CRM/Contact (4), Intelligence Core (4), Intelligence Objects (5), Opportunity/Pursuit (3), Knowledge/Capability (5), Email/Sequences (9), Data Import/Quality (9), Validation/Feedback (5), Connectors (2), Human Intelligence (3), AI/Reasoning (7), Revenue Intelligence (3), Job/Workflow (2), AI Cost/Audit (5), Learning (2), Pipeline/Fusion (2), Account Strategy (3), Auth/User (3), System (2), and External/Monitoring (2). The Company model acts as the central hub with approximately 35 explicit back-relation fields, making it the most connected model in the schema.

5.2 Index Coverage

The 372 indexes across 92 models provide exceptional query optimization. Notable compound indexes include CompanySignal with status+expiresAt and companyId+signalType+createdAt, Company with priorityTier and accountPriorityScore (descending sort), and Evidence with companyId+status+createdAt. However, 38 models (41%) have zero indexes, including high-traffic entities like AccountScore, CompanyResearchCard, IntelligenceObject, and KnowledgeChunk. Queries against these unindexed models will degrade linearly with data volume.

5.3 Data Integrity Concerns

The most significant risk is the 97 cascade delete relations. A single Company.delete() would cascade into 35+ related models, potentially destroying thousands of records across signals, evidence, contacts, notes, timelines, intelligence objects, knowledge entries, associations, alerts, learning events, and more. Only 3 SetNull relations exist. An additional concern is the 18 String-stored JSON fields that cannot leverage Prisma JSON filtering operators, requiring manual JSON.parse/stringify at the application layer. Approximately 15 fields use raw String types where enums should be used for type safety at the database level.

5.4 Key Findings

ID	Severity	Finding	Evidence
DB1	CRITICAL	97 cascade deletes on Company	Single delete destroys entire graph
DB2	HIGH	18 String-stored JSON fields	Cannot use Prisma JSON operators
DB3	HIGH	38 models with zero indexes	Queries degrade with data volume
DB4	MEDIUM	God model: Company 35+ relations	Every query path touches Company
DB5	MEDIUM	No connection pool configuration	Could exhaust Neon limits
DB6	LOW	Single migration file only	No incremental migration history

Table 5: Database Findings

6. Data Intelligence Pipeline

The data intelligence pipeline has two parallel entry points: a Data Import pipeline for CSV/Excel files and an Intelligence Sources pipeline with 4 connectors (CSV, Excel, RSS, Website) plus an internal memory bridge that connects 6 CRM data sources. Both paths converge through company resolution and evidence classification into CompanySignal records, which feed into opportunity recommendations, AI insights, and action artifacts.

6.1 Data Import

The platform has a dual import pipeline: a legacy pipeline creating ImportBatch records and a newer intelligence engine pipeline creating DataUpload and UploadRow records with a bridge at commit time. The newer engine implements a 6-step process: analyze file headers via regex column detection, create upload job, process chunks with validation/normalization/dedup/quality scoring, review rows with user corrections, apply corrections, and commit with intelligence activation. Column detection is fully DB-configurable with zero hardcoded patterns. Deduplication uses three strategies: exact email match, domain+name similarity (60% threshold), and company name fuzzy match with legal suffix stripping.

6.2 Intelligence Sources

The intelligence sources module comprises 55 files across connectors, signal creation, evidence classification, freshness management, job queuing, and monitoring. The internal memory connector bridges 6 CRM data sources (CompanyNote, ContactNote, EmailEvent, CompanyTimelineEvent, HumanIntelligenceInbox, AccountStrategy) into the intelligence pipeline with confidence weights ranging from 0.70 to 0.95. Signal creation supports 12 signal types with regex-based classification and deduplication. The freshness system applies exponential decay with a 180-day max age and 0.5% daily decay rate. The job queue is entirely in-memory with sequential processing, representing a single point of failure for serverless deployments.

6.3 Key Findings

ID	Severity	Finding	Evidence
P1	HIGH	In-memory job queue	Jobs lost on serverless restart
P2	MEDIUM	Dual import pipelines	ImportBatch vs DataUpload confusion
P3	MEDIUM	Cross-chunk dedup gap	validateRows not used in engine
P4	MEDIUM	Sequential signal creation	No batch createMany
P5	LOW	Missing document connector	SourceType supports it but no impl

Table 6: Data Pipeline Findings

7. Backend API Architecture

The backend API comprises 239 route files across approximately 66 domain directories, totaling 43,438 lines of code. The API follows Next.js 16 App Router conventions with route handlers at `src/app/api/[domain]/route.ts`. Authentication is implemented via a dual-layer defense-in-depth approach: an Edge proxy checks for session cookie presence, and route-level guards validate session tokens against the database. Input validation uses Zod schemas for all CRUD operations, and the AI governance layer (governedAICall) wraps all AI-generating endpoints.

7.1 API Patterns

Three coexisting error response patterns create inconsistency: `apiError/apiSuccess` with success boolean and timestamp, bare `NextResponse.json` with error only, and the Intelligence API envelope with code and details. Similarly, success responses use four different formats. This forces frontend consumers to handle multiple response shapes. The alignment route at 1,027 lines is a monolithic handler that should be decomposed. Input validation is well-adopted for POST bodies via Zod but inconsistently applied to GET query parameters, where `sortBy`, `page`, and `limit` are often manually parsed without schema validation.

7.2 Authentication Coverage

Of 239 routes, 220 (92%) have route-level `checkApiAuth()` guards. The 19 routes without route-level auth are all legitimately public (health, auth endpoints, webhooks, tracking pixels) with one critical exception: `/api/ai/evaluation` has no authentication mechanism at all. This route exposes both read (evaluation stats) and write (trigger evaluations) operations to anyone with a session cookie, even an expired one. Mass assignment is generally well-protected through explicit field whitelisting on PATCH routes, though 7 fields on the companies PATCH endpoint bypass Zod validation, allowing arbitrary values for `intelligenceScore`, `assignedTo`, and other sensitive fields.

7.3 Missing Infrastructure

No API versioning strategy exists (no `/v1/` prefixes, no version headers), meaning any breaking change affects all clients simultaneously. Rate limiting is implemented via in-memory Maps that are not production-safe for multi-instance deployments. The API documentation (`API_REFERENCE.md`) covers 224 routes but is stale, with 15 newer routes undocumented. CSRF protection is implemented at the proxy level with constant-time comparison, which is well-designed.

7.4 Key Findings

ID	Severity	Finding	Evidence
API1	CRITICAL	<code>/api/ai/evaluation</code> : no auth	Read/write access without session validation
API2	HIGH	Companies PATCH: 7 fields bypass Zod	<code>intelligenceScore</code> , <code>assignedTo</code> unprotected
API3	HIGH	In-memory rate limiting	Not multi-instance safe
API4	MEDIUM	3 inconsistent error formats	Client complexity from multiple shapes
API5	MEDIUM	No API versioning	Breaking changes affect all clients
API6	LOW	API docs stale (224 vs 239)	15 routes undocumented

Table 7: API Findings

8. UI/UX Analysis

The UI architecture uses an SPA-style pattern within Next.js App Router. A single page route renders the application shell, with navigation handled entirely client-side via a Zustand store (89 possible activeView values) mapped through a screen-map.tsx to lazy-loaded React components wrapped with per-screen ErrorBoundary and Suspense. The navigation config defines 5 sections (Intelligence, Revenue, Knowledge, Data, Operations) with 13 visible nav items. The Intelligence OS layer introduces 8 newer screens that overlap functionality with 64 legacy screens, creating a maintenance burden.

8.1 Component Quality

The platform includes 68 screen components totaling 52,451 lines of code, 14 enterprise shared components, 9 shared UI components, and 26 Intelligence OS components. Four screens exceed 2,000 lines: company-profile-screen (2,450), knowledge-library (2,396), settings (2,322), and capability (2,067). These giant monolithic components should be decomposed into 10+ sub-components each for maintainability. The design system provides reusable primitives (EmptyState, ScoreGauge, DataTable, LoadingState, ErrorState) with proper TypeScript interfaces.

8.2 Accessibility

Accessibility implementation is severely deficient. Across 52,451 lines of screen code, only 56 ARIA attributes and 14 role attributes were found, averaging 0.001 per line of code. Only 15% of screens (10 of 68) have keyboard handlers. Enterprise customers, especially those in government or regulated industries, will likely fail accessibility audits. This is a high-priority gap for any enterprise deployment scenario.

8.3 Theme & Responsive

The platform has a dark-first design with CSS variables defined in :root, but enterprise-theme.ts uses hardcoded light colors, creating an inconsistent hybrid. There is no theme toggle. Responsive design is well-implemented with 446 breakpoint references across screens (average 6.5 per screen) using progressive disclosure from 1-column mobile to multi-column desktop layouts. Loading states are excellent with 772 skeleton/isLoading references, while error states are adequate through the per-screen ErrorBoundary pattern.

8.4 Key Findings

ID	Severity	Finding	Evidence
UX1	HIGH	Accessibility severely lacking	70 ARIA attrs across 52K LOC
UX2	MEDIUM	4 giant components >2000 LOC	company-profile at 2450 lines
UX3	MEDIUM	Theme inconsistency	Dark CSS vars + hardcoded light colors
UX4	MEDIUM	No client-side auth redirect	AuthProvider is no-op
UX5	LOW	Legacy/Intel OS duplication	64 legacy + 8 new overlapping screens

Table 8: UI/UX Findings

9. Security Analysis

The security analysis reveals a platform with strong authentication fundamentals but critical infrastructure gaps. The OTP-based authentication system uses cryptographically sound session tokens (32-byte random values), PBKDF2 password hashing with 100k iterations, SHA-256 hashed OTP storage, timing-attack-resistant comparisons, and rolling 30-day session expiry. AI governance is excellent with a custom ESLint rule preventing ungoverned LLM calls. However, the most critical finding is that the Edge middleware file (`src/middleware.ts`) does not exist, meaning security headers are never applied and CSRF tokens are never injected, rendering the entire CSRF protection system non-functional.

9.1 Authentication

Session management is solid with opaque tokens stored in the database, `httpOnly/secure/sameSite=lax` cookies, and expired session cleanup. The OTP system uses 4-byte random values with 10-minute expiry, 5-attempt limit, and dual-layer rate limiting. A timing vulnerability exists in the cookie-hash verification path which uses non-constant-time string comparison. The dev OTP bypass mechanism (`ALLOW_DEV_OTP=true`) is guarded by `NODE_ENV !== production`, but `NODE_ENV` misconfiguration could expose OTP codes in API responses. The `API_KEY_ENCRYPTION_KEY` for AES-256-GCM encryption of AI provider keys is not documented in `.env.example` and silently falls back to plaintext storage if unset.

9.2 Critical Security Gaps

The most severe security finding is the missing Edge middleware. The `next.config.ts` comments that security headers are "applied via Edge middleware (`src/middleware.ts`)" but this file does not exist. This means: (1) No security headers (`X-Content-Type-Options`, `X-Frame-Options`, `HSTS`, `CSP`) are sent on any response, (2) CSRF tokens are never injected as cookies, and (3) The entire CSRF protection infrastructure, while well-implemented in code, is completely non-functional in production. The `fetchApi` client utility does not send the `x-csrf-token` header, compounding the issue. Additionally, input sanitization relies on a 6-line regex HTML stripper rather than a proper library like `DOMPurify`, creating XSS risk for any user-generated content rendered as HTML.

9.3 Key Findings

ID	Severity	Finding	Evidence
S1	CRITICAL	Missing Edge middleware	No security headers, CSRF non-functional
S2	CRITICAL	<code>/api/ai/evaluation</code> : no auth	Unrestricted AI operation access
S3	HIGH	CSRF protection non-functional	No token injection, no client header
S4	HIGH	Input sanitization inadequate	6-line regex, not <code>DOMPurify</code>
S5	HIGH	Dev OTP bypass production risk	<code>ALLOW_DEV_OTP</code> + <code>NODE_ENV</code> misconfig
S6	MEDIUM	API key encryption key undocumented	Silent plaintext fallback
S7	MEDIUM	In-memory rate limiting	Per-instance only, not global
S8	MEDIUM	CSP allows <code>unsafe-inline</code>	Weakens XSS protection

Table 9: Security Findings

10. Performance & Scalability

Performance and scalability analysis reveals that the platform is designed for single-instance deployment and faces significant challenges for horizontal scaling or serverless environments. The most critical concern is that 10+ modules maintain in-memory state (knowledge graph, AI memory, hybrid retrieval, rate limiting, batch progress, OTP cache, activation history, event bus, reasoning state, vector index) that is lost on cold starts and inconsistent across multiple instances.

10.1 Database Performance

The 372 indexes provide good read performance, but 38 of 92 models have zero indexes, and no explicit connection pool configuration exists for Neon PostgreSQL. N+1 query risks exist in the intelligence activation pipeline where sequential `addNode/addEdge` calls process each contact serially (capped at 10). Offset-based pagination is used exclusively with no cursor-based alternative for large datasets. No query timeout configuration means long-running queries could hang indefinitely.

10.2 Frontend Performance

All image optimization is disabled (`images: { unoptimized: true }` in `next.config.ts`), likely to avoid Vercel Image Optimization costs but impacting user-perceived performance. Zero dynamic imports are used across all 55+ screen components, meaning the initial JavaScript bundle includes all screens regardless of which ones the user navigates to. React Query caching uses a 30-second stale time with `refetchOnWindowFocus` disabled, which is adequate. The AI cache layer is database-backed (AICache table) with SHA-256 key generation and TTL-based expiry, but the `prune()` function is never called automatically, leading to unbounded table growth.

10.3 Deployment

The deployment configuration supports three targets: Docker (multi-stage build with alpine, non-root user, health check), Render (single web service, starter plan, daily cron), and Vercel (Mumbai region, no edge deployment, daily cron). The architecture is single-instance with no Redis, no worker processes, no message queue, and no horizontal scaling support. Docker Compose includes only the app and PostgreSQL. The LLM timeout uses `setTimeout` without `AbortController`, meaning HTTP requests are not actually cancelled when the timeout fires.

10.4 Key Findings

ID	Severity	Finding	Evidence
P1	CRITICAL	10+ modules with in-memory state	Lost on cold start, not shared
P2	HIGH	No CI/CD pipeline exists	Tests never run automatically
P3	HIGH	38 models with zero indexes	Queries degrade with scale
P4	MEDIUM	<code>images: { unoptimized: true }</code>	All image optimization disabled
P5	MEDIUM	Zero dynamic imports	All 55+ screens in initial bundle
P6	MEDIUM	No <code>AbortController</code> on LLM timeout	Connections not cancelled
P7	LOW	AI cache <code>prune()</code> never called	Unbounded table growth

Table 10: Performance Findings

11. Testing

The testing infrastructure uses Vitest 4.1.10 with jsdom environment, @vitest/coverage-v8 for code coverage, and @testing-library/react for component testing. A total of 83 test files exist containing 42,656 lines of test code, yielding an approximate 20% test-to-code ratio. However, 17 test files (20.5%) are excluded from CI due to stale code references, and 128 of 194 lib modules (66%) have no corresponding tests. No CI/CD pipeline exists, no GitHub Actions workflows were found, and there are zero true integration or E2E tests despite an e2e-business-journey.test.ts file existing.

11.1 Test Quality

The WI-17 test suite (4 test files, 3,296 lines) demonstrates high-quality test writing with proper vi.mock() patterns, vi.resetAllMocks() for isolation, comprehensive edge case coverage, and structured test descriptions. Security tests cover 7+ files including auth, OTP, blocking, admin routes, and hygiene. However, all database interactions are mocked (no integration tests with a real database), and the test quality concern is that heavy mocking verifies function call patterns rather than actual data transformations or query correctness. The api-routes.test.ts file primarily tests utility functions rather than actual API route HTTP request/response cycles.

11.2 Critical Gaps

The following high-criticality modules have zero test coverage: ai-governance.ts (controls all AI calls), ai-knowledge-graph.ts (entire graph engine), ai-memory.ts (all AI memory), ai-hybrid-retrieval.ts (retrieval engine), ai-unified-confidence.ts (confidence scoring), llm-client.ts (all LLM calls), intelligence-pipeline.ts (core enrichment), workflow-engine/queue.ts and processor.ts (job processing), and rate-limit.ts. The absence of tests for ai-governance.ts is particularly concerning as it controls the cost, quality, and safety gates for all AI generation across the platform.

11.3 Key Findings

ID	Severity	Finding	Evidence
T1	CRITICAL	No CI/CD pipeline	Tests never run automatically
T2	HIGH	66% modules have no tests	128 of 194 lib modules untested
T3	HIGH	AI governance untested	Controls all AI cost/quality gates
T4	HIGH	17 test files excluded	121 assertions disabled
T5	MEDIUM	Zero integration/E2E tests	All tests use mocked DB
T6	MEDIUM	No coverage reporting	Tool installed but unused

Table 11: Testing Findings

12. Product Readiness Scoring

Each of the 10 audit categories is scored on a 0-10 scale based on functionality completeness, quality of implementation, production readiness, and alignment with enterprise standards. The overall product readiness score is the weighted average, with Security and Testing receiving higher weights due to their blocking nature for enterprise deployment.

Category	Score / 10	Status
Architecture	7.5	Clean dependency graph, well-unified LLM client, but flat lib/ and dead middleware code
AI Intelligence	7.0	Sophisticated multi-layer architecture, excellent governance, but entirely in-memory
Database	7.0	92 models, 372 indexes, but god model pattern and cascade delete risks
Data Pipeline	6.5	Comprehensive connectors and intelligence sources, dual-path complexity
Backend API	5.5	239 routes with good auth coverage, but 3 inconsistent response formats
UI/UX	6.0	Feature-rich with 68 screens, but accessibility severely lacking
Security	3.5	Strong auth fundamentals, but missing middleware = no headers, no CSRF
Performance	4.5	In-memory state prevents scaling, no CI/CD, no dynamic imports
Testing	4.0	Good WI-17 tests, but 66% modules untested, no CI, no E2E
Product Ready	5.8	Advanced MVP with sophisticated AI, needs production hardening

Table 12: Detailed Scoring Breakdown (out of 10)

12.1 Verdict: B) Advanced MVP

DeepMindQ is assessed as an **Advanced MVP** (Score: 5.8/10). The platform demonstrates exceptional AI architecture with a sophisticated multi-layer intelligence system, comprehensive database design, and a feature-rich UI. The 213,617 lines of code across 676 files represent a substantial engineering effort. However, three blocking issues prevent enterprise-beta classification: (1) the missing security middleware means no production security headers or CSRF protection, (2) the in-memory AI systems lose all institutional knowledge on serverless cold starts, and (3) the absence of CI/CD means no automated quality gates. These are all addressable in a focused WI-18 production hardening phase.

A) Prototype	1-3	Basic functionality, incomplete flows, significant bugs
--------------	-----	---

B) Advanced MVP	4-6	Working end-to-end, needs production hardening
C) Enterprise-Ready Beta	7-8	Production-deployable with monitoring
D) Production Enterprise	9-10	Fully hardened, monitored, scalable

Table 13: Product Maturity Classification (DeepMindQ = B)

13. Final Recommendations

Recommendations are organized into three priority tiers. P0 items are blocking issues that must be resolved before any enterprise deployment. P1 items are high-priority improvements that should be addressed in the next development phase (WI-18). P2 items are medium-priority improvements for subsequent phases. This prioritization is designed to serve as the master roadmap from WI-18 onwards.

13.1 P0: Blocking (Must Fix Before Enterprise Deployment)

P0-1: Create Edge Middleware (src/middleware.ts) — This is the single most impactful security fix. The middleware must inject CSRF tokens, apply security headers (X-Content-Type-Options, X-Frame-Options, HSTS, CSP), and validate session cookies at the Edge level. Without this, the platform has no production security headers and no functional CSRF protection. Estimated effort: 1-2 days.

P0-2: Add checkApiAuth() to /api/ai/evaluation — This route exposes AI evaluation operations without any authentication. Adding the standard checkApiAuth() guard is a one-line fix with immediate security impact. Estimated effort: 30 minutes.

P0-3: Persist AI State to Database — The knowledge graph, AI memory, and hybrid retrieval index must be backed by database persistence to survive serverless cold starts. This is the most architecturally significant P0 item. Options include: (a) Prisma-backed graph tables with in-memory cache layer, (b) Redis for session-level persistence with periodic DB sync, or (c) SQLite-based local persistence with periodic sync. Estimated effort: 2-3 weeks for the full persistence layer.

P0-4: Implement CI/CD Pipeline — Set up GitHub Actions with at minimum: lint, type-check, test execution on PR, and build verification. This provides automated quality gates that currently do not exist. Estimated effort: 1-2 days.

13.2 P1: High Priority (WI-18 Phase)

- Migrate rate limiting to Redis-backed store for multi-instance safety
- Extend updateCompanySchema to include all updatable fields (close mass-assignment gap)
- Add DOMPurify or equivalent server-side HTML sanitizer for user-generated content
- Document API_KEY_ENCRYPTION_KEY in .env.example and remove silent plaintext fallback
- Standardize API response format to single envelope (recommend apiSuccess/apiError pattern)
- Decompose 4 giant screen components into sub-components
- Add Zod validation for all GET query parameters (sortBy, page, limit)
- Fix ModelRouter tier routing (orderedChain computed but unused)
- Implement accessibility basics: ARIA labels on tables, keyboard navigation, focus indicators
- Add API versioning (/v1/ prefix) before any external API consumers
- Fix extractJSON() greedy regex to use balanced-brace parser
- Add connection pool configuration for Neon PostgreSQL

13.3 P2: Medium Priority (Post-WI-18)

- Replace hedging detection with confidence-aware scoring in quality gates
- Upgrade from TF-IDF to neural embeddings for semantic search
- Add cursor-based pagination for large datasets
- Implement dynamic imports for screen components to reduce initial bundle

- Consolidate legacy screens with Intelligence OS layer (remove duplication)
- Add proper theme system with dark/light toggle support
- Implement AbortController for LLM timeout cancellation
- Create integration tests with real database (not all-mocked)
- Add Playwright E2E tests for critical user flows
- Schedule AI cache prune() via cron job
- Restructure cascade deletes: add Restrict on critical junction models
- Convert 18 String-stored JSON fields to proper Json type
- Add missing enums for 15+ String fields that should be typed
- Unify dual import pipelines (legacy ImportBatch + new DataUpload)
- Add per-chunk cross-row deduplication in data import engine

13.4 Architecture Strengths to Preserve

While this audit identifies many areas for improvement, several architectural strengths should be explicitly preserved and built upon: (1) The non-throwing design philosophy across all AI modules provides excellent resilience, (2) The comprehensive AI governance framework with ESLint enforcement is enterprise-grade, (3) The clean dependency graph with zero circular dependencies enables safe refactoring, (4) The unified LLM client with quality gates is well-abstracted, (5) The AES-256-GCM encryption for API keys with base URL allowlisting is security-conscious, (6) The 372-index database schema provides exceptional read performance, and (7) The per-screen ErrorBoundary pattern prevents cascade failures in the UI layer.

End of Audit Report. All findings are based on actual codebase evidence. This document should serve as the master roadmap from WI-18 onwards.